

3A) On veut modifier m-pos de Player suivant une méthode de Terrain

1. m-pos est privé $\rightarrow$ public
ou getter/setter

⚠ Brise l'encapsulation: complexifie la compréhension et le debug de m-pos

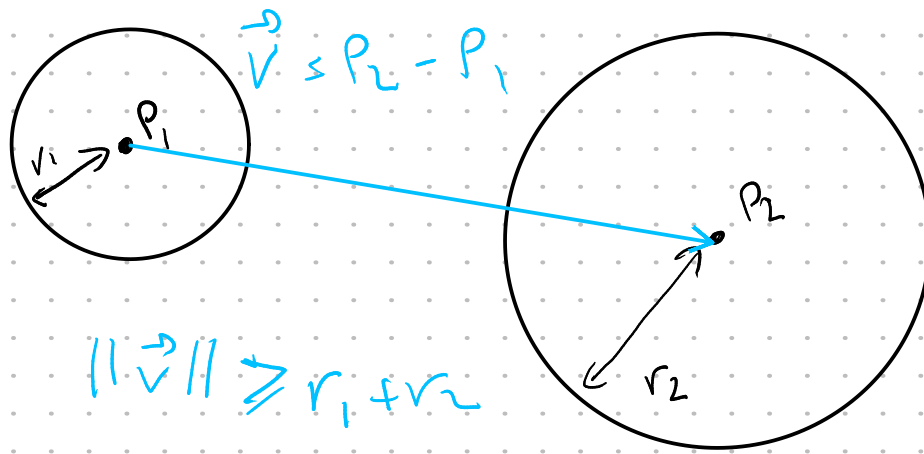
2. Player peut garder une référence / pointeur vers le Terrain



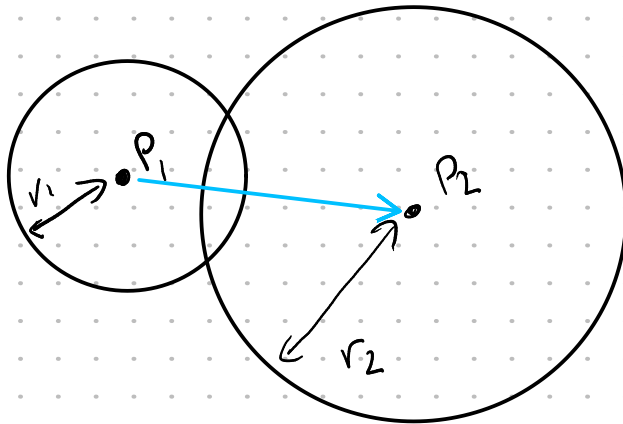
3. Méthode dédiée qui a une référence de Terrain seulement le temps du traitement

😊 Pas de couplage entre la durée de vie des Player et du Terrain

3.6

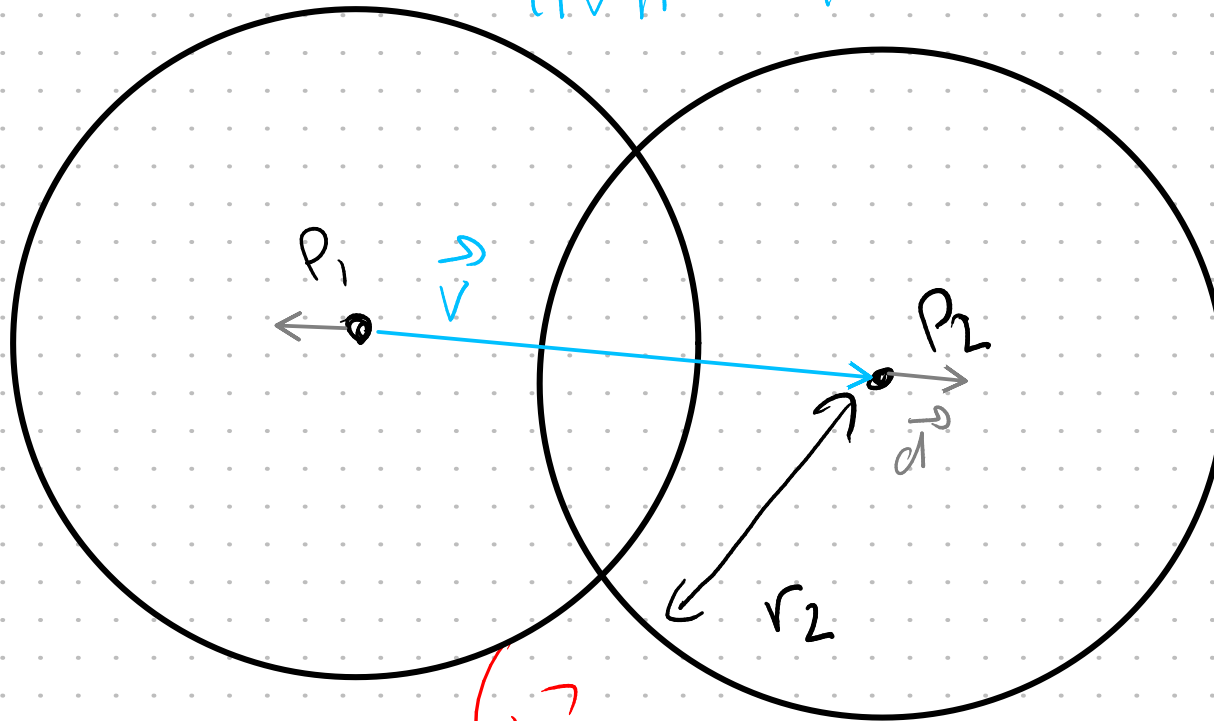


$$\|\vec{v}\| < r_1 + r_2$$



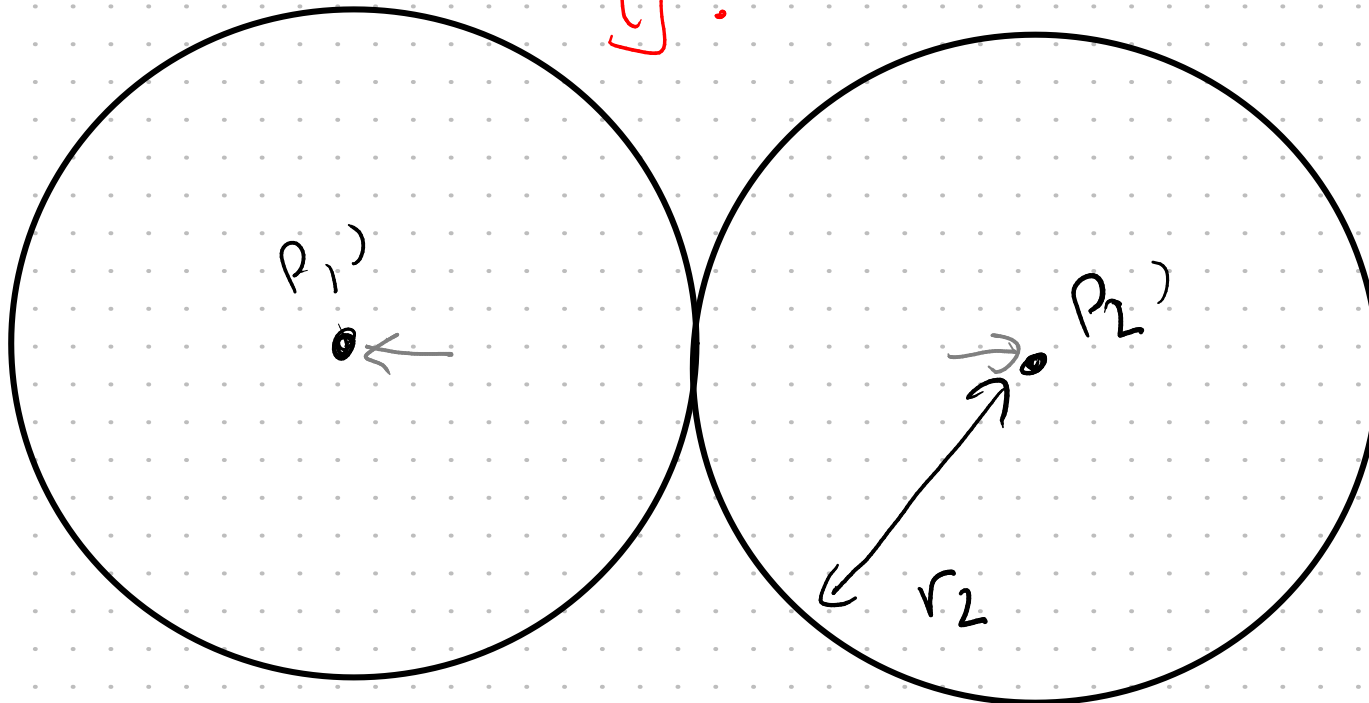
3D

$$\|\vec{v}\| < r_1 + r_2$$



$$\|2\vec{d} + \vec{v}\| = 2r$$

$\vec{d} = \alpha \vec{v}$
 $\rightarrow \vec{d}$ est dans la même direction que $\vec{v}$



$$P_1' = P_1 - \vec{d}$$

$$P_2' = P_2 + \vec{d}$$

$$\|2\vec{d} + \vec{v}\| = 2r$$

$$\vec{d} = \alpha \vec{v}$$

→ $\vec{d}$ est dans la même direction que $\vec{v}$

$$\begin{cases} \|2\alpha \vec{v} + \vec{v}\| = 2r \\ \|(2\alpha + 1)\vec{v}\| = 2r \\ (2\alpha + 1)\|\vec{v}\| = 2r \end{cases}$$

$$2\alpha + 1 = \frac{2r}{\|\vec{v}\|}$$

$$2\alpha = \frac{2r - \|\vec{v}\|}{\|\vec{v}\|}$$

$$\alpha = \frac{2r - \|\vec{v}\|}{2\|\vec{v}\|}$$

$$\vec{d} = \alpha \vec{v} = \frac{2r - \|\vec{v}\|}{2\|\vec{v}\|} \vec{v}$$

$$= \underbrace{(2r - \|\vec{v}\|)}_{\text{Distance idéale - distance réelle}} \times \underbrace{\frac{\vec{v}}{\|\vec{v}\|}}_{\text{Vecteur unitaire de la direction de } \vec{v}} \times \underbrace{\frac{1}{2}}_{\text{car on l'applique sur les 2 extrémités}}$$

Distance idéale - distance réelle

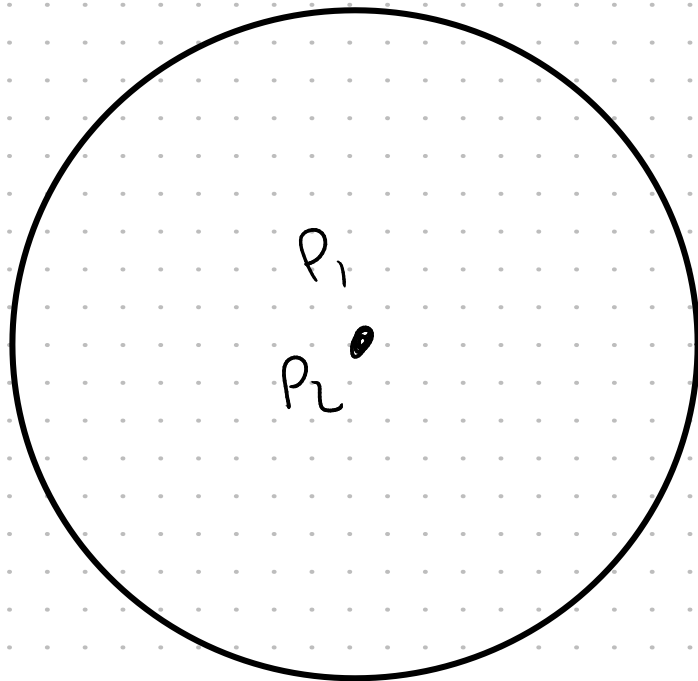
$$\vec{d} = \alpha \vec{v} = \frac{2v - \|\vec{v}\|}{2\|\vec{v}\|} \vec{v}$$

Factor

division par zéro?

$$\|\vec{v}\| = 0$$

$$P_1 = P_2$$



→ Cas particuliers, car on ne peut pas choisir une direction de déplacement

```
for (Player& player : m_players)
    for (Player& player2 : m_players)
        // Resoudre la collision entre player et player 2
        // et modifier 'm_pos' en conséquence
```

Privé → Méthode

player.Resolve(player2) ↗
player2.Resolve(player) ↖

Resolve() est symétrique (les 2 instances sont traitées de manière égale)

static void Resolve(player, player2)

→ Méthode non-reliée à une instance implicite

→ Toutes les données nécessaires sont prises en argument

```

for (Player& player : m_players)
    for (Player& player2 : m_players)
        Player::ResolveCollision(player, player2);

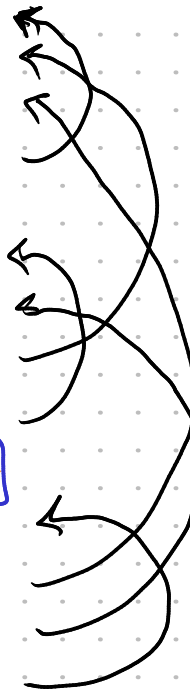
```

$m_players : [j_0, j_1, j_2, j_3]$

j_0 [j_0 C
 j_1 C
 j_2 C
 j_3 C
 j_1 [j_0 C
 j_1 C
 j_2 C
 j_3 C
 j_2 [j_0 C
 j_1 C
 j_2 C
 j_3 C
 j_3 [j_0 C
 j_1 C
 j_2 C
 j_3 C

~~x~~ $j_0 \times j_0$
 $\rightarrow j_0 \times j_1$
 $\rightarrow j_0 \times j_2$
 $\rightarrow j_0 \times j_3$
~~x~~ $j_1 \times j_0$
 $\times$ $j_1 \times j_1$
 $\rightarrow j_1 \times j_2$
 $\rightarrow j_1 \times j_3$
~~x~~ $j_2 \times j_0$
~~x~~ $j_2 \times j_1$
 $\times$ $j_2 \times j_2$
 $\rightarrow j_2 \times j_3$
~~x~~ $j_3 \times j_0$
~~x~~ $j_3 \times j_1$
~~x~~ $j_3 \times j_2$
 $\times$ $j_3 \times j_3$

1. On checke les collisions avec soi-même



y_0



y_1

y_2

y_3

y_1



y_2

y_3

y_2



y_3

y_3




```
static void ResolveCollision(Animal& a, Animal& b);
```

→ Dans Player et Animal (H)

```
40 void dino::Animal::ResolveCollision(Animal& a, Animal& b)
41 {
42     // On ne peut pas accéder à m_pos car méthode statique = pas d'état implicite
43     // On peut accéder à a.m_pos et b.m_pos car en paramètre, et que la méthode peut accéder à
44     // l'état privé
45
46     Vec2 dist{b.m_pos.x - a.m_pos.x, b.m_pos.y - a.m_pos.y};
47     float distnorm = sqrtf(dist.x * dist.x + dist.y * dist.y);
48
49     if (distnorm > 0 && distnorm < 16)
50     {
51         float factor = (16 - distnorm) / (2 * distnorm);
52         Vec2 depl{dist.x * factor, dist.y * factor};
53         a.m_pos.x -= depl.x;
54         a.m_pos.y -= depl.y;
55         b.m_pos.x += depl.x;
56         b.m_pos.y += depl.y;
57     }
58 }
```

→ Dans Player et Animal (C++)

```
for (int i = 0; i < m_players.size(); ++i)
    for (int j = i + 1; j < m_players.size(); ++j)
        Player::ResolveCollision(m_players[i], m_players[j]);
```

```
for (int i = 0; i < m_animals.size(); ++i)
    for (int j = i + 1; j < m_animals.size(); ++j)
        Animal::ResolveCollision(m_animals[i], m_animals[j]);
```

→ à bande copié/collee

Cette fonction est copié-collée

→ Prend des Animal ou Player

```
40 void dino::Animal::ResolveCollision(Animal& a, Animal& b)
41 {
42     // On ne peut pas accéder à m_pos car méthode statique = pas d'état implicite
43     // On peut accéder à a.m_pos et b.m_pos car en paramètre, et que la méthode peut accéder à
44     // l'état privé
45
46     Vec2 dist{b.m_pos.x - a.m_pos.x, b.m_pos.y - a.m_pos.y};
47     float distnorm = sqrtf(dist.x * dist.x + dist.y * dist.y);
48
49     if (distnorm > 0 && distnorm < 16)
50     {
51         float factor = (16 - distnorm) / (2 * distnorm);
52         Vec2 depl{dist.x * factor, dist.y * factor};
53         a.m_pos.x -= depl.x;
54         a.m_pos.y -= depl.y;
55         b.m_pos.x += depl.x;
56         b.m_pos.y += depl.y;
57     }
58 }
```

On lit et écrit m_pos
→ on se fiche des autres
variables membres des classes
Animal et player

→ Créer une classe qui regroupe ces points communs

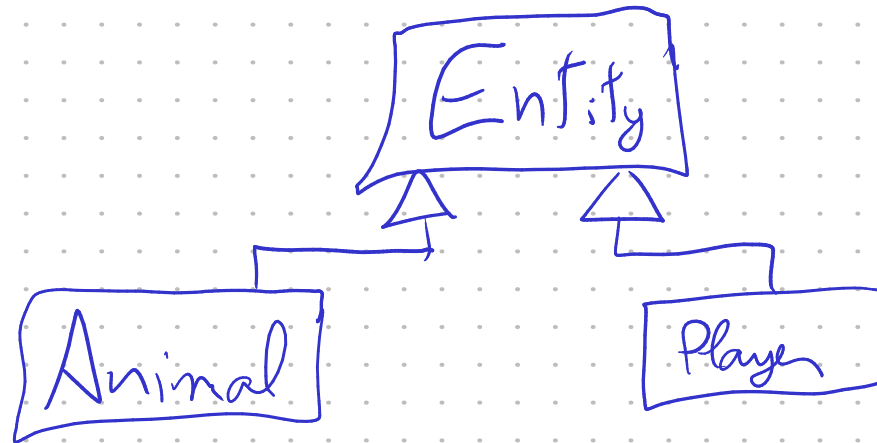
```
class Entity
{
public:
    static void ResolveCollision(Entity& a, Entity& b);

private:
    Vec2 m_pos;
};
```

```
class Animal : public Entity
```

<< Un Animal est un cas particulier d'Entity >>

→ Héritage car Animal hérite des comportements d'Entity
et Animal hérite de l'état d'Entity



Classe de base
= classe parents

Classes dérivées
= classes enfants

```
// Représente un animal.  
class Animal : public Entity
```

```
{  
public:  
    /// Initialise l'animal avec un type au hasard  
    Animal(Vec2 pos, double absTime, jv::gpu::Text
```

```
    /// Constructeur d'Animal à partir d'un autre  
    ///  
    /// "Animal a = b;" ou "Animal a{b};" (on cons  
    Animal(const Animal& other) = delete;
```

```
    /// Opérateur d'assignement par copie  
    ///  
    /// Animal a, b;  
    /// a = b; // Pas un constructeur, car 'a' et  
    Animal& operator=(const Animal& b) = delete;
```

```
    /// Déplace l'animal et met à jour son animati  
    void Update(double absTime, float deltaTime);
```

```
    /// S'assurer que la position du joueur reste  
    void CheckTerrain(Terrain const& terrain);
```

```
    /// Affiche l'animal  
    void Draw() const;
```

```
    /// Détruit les ressources associées à l'anim  
    ~Animal();
```

```
// Représente un joueur.  
class Player : public Entity  
{  
public:
```

```
    /// Initialise le joueur avec un type au hasa  
    Player(  
        Vec2 pos, int32_t idxPlayer, jv::input::G
```

```
    /// Constructeur d'un Player à partir d'un au  
    ///  
    /// "Player a = b;" ou "Player a{b};" (on con  
    Player(const Player& other) = delete;
```

```
    /// Opérateur d'assignement par copie  
    ///  
    /// Player a, b;  
    /// a = b; // Pas un constructeur, car 'a' et  
    Player& operator=(const Player& b) = delete;
```

```
    /// Déplace le joueur et met à jour son anima  
    void Update(double absTime, float deltaTime);
```

```
    /// S'assurer que la position du joueur reste  
    void CheckTerrain(Terrain const& terrain);
```

```
    /// Affiche le joueur  
    void Draw() const;
```

```
    /// Détruit les ressources associées au joueu  
    ~Player();
```

Constructeurs, ne sont pas en combinaison

= delete

Code très différent

Presque pareil

```
void dino::Player::CheckTerrain(Terrain const& terrain)
{
    m_pos = terrain.ClampPos(m_pos);
}
```

← presque pareil

```
void dino::Animal::CheckTerrain(Terrain const& terrain)
{
    Vec2 oldPos = m_pos;
    m_pos = terrain.ClampPos(m_pos);
    if (m_pos.x != oldPos.x || m_pos.y != oldPos.y)
        m_dir = jv::util::RandomRotate({1, 0}, 0, 360);
}
```

↑
Mais réaction en plus

→ Mettre en commun ce qui est pareil dans Entity

→ Laisser les classes dérivées surcharger la réaction

```
class Entity
```

```
{
```

```
    virtual void _ReactTerrain();
```

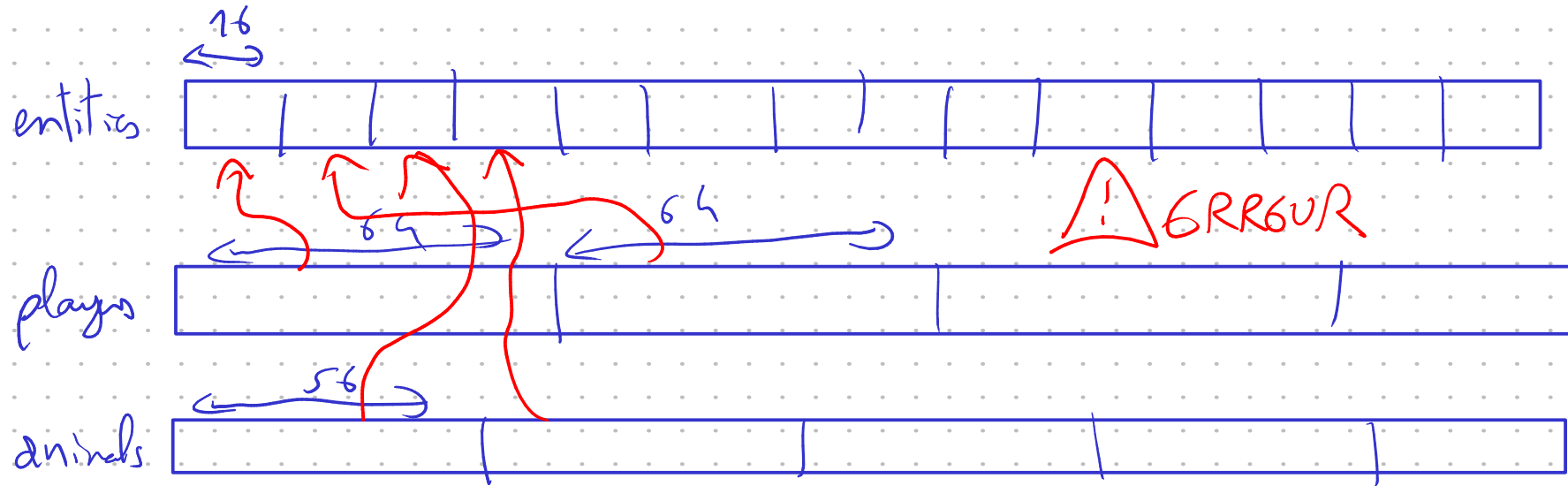
```
void dino::Entity::CheckTerrain(Terrain const& terrain)
{
    Vec2 oldPos = m_pos;
    m_pos = terrain.ClampPos(m_pos);
    if (m_pos.x != oldPos.x || m_pos.y != oldPos.y)
        _ReactTerrain();
}
```

```
void dino::Entity::_ReactTerrain()
{
}
```

```
void dino::Animal::_ReactTerrain()
{
    m_dir = jv::util::RandomRotate({1, 0}, 0, 360);
}
```

```
std::vector<Entity> entities;
for (Player& player : m_players)
    entities.push_back(player);
for (Animal& animal : m_animals)
    entities.push_back(animal);
```

type	value
sizeof(Entity)	16
sizeof(Player)	64
sizeof(Animal)	56



1. Tableau commun avec toutes les entités

```
m_entities.resize(0);  
for (Player& player : m_players)  
    m_entities.push_back(&player);  
for (Animal& animal : m_animals)  
    m_entities.push_back(&animal);
```

2. Ordre suivant la position Y

```
std::sort(m_entities.begin(), m_entities.end(), Entity::OrderByPosY);
```

→ Méthode car m-est n'est pas public

3. Dessiner dans l'ordre de m-entités

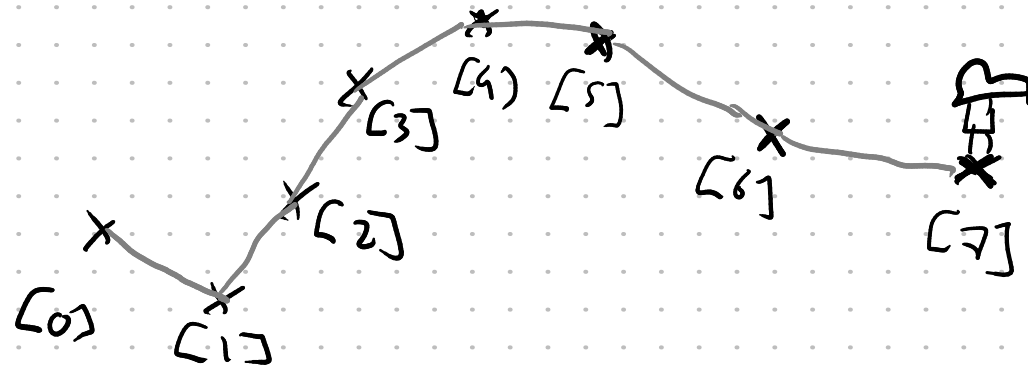
```
void dino::Scene::Draw() const  
{  
    m_Terrain.Draw();  
  
    for (Entity const* entity : m_entities)  
        entity->Draw();  
}
```

Code différent suivant Animal et Player
→ Méthode virtuelle

```
class Entity  
{  
public:  
    static void ResolveCollision(Entity& a, Entity& b);  
    static bool OrderByPosY(Entity const* a, Entity const* b);  
  
    virtual void Draw() const = 0;
```

les classes dérivées
DOIVENT spécifier un
comportement (pas de défaut)

LASSO = Suite de points = vector <Vec2>



lasso.size() == 8

dino_draw_utils.h

```
/// Produit un dessin contenant une liste de segments, tous reliés.  
/// Les sommets doivent être dessinés avec XDino_TEXID_WHITE.  
///  
/// @param out Destination dans laquelle sont ajoutés les sommets.  
/// @param points Liste de points par lesquels la polygone passe.  
/// @param width Epaisseur du trait, en pixels.  
/// @param color Couleur du trait.  
void GenVertices_Polyline(std::vector<jv::gpu::Vertex>& out, std::vector<Vec2> const& points,  
    float width, jv::util::Color color = Color_WHITE);
```

└ Largeur

SORTIR
Vertex

Liste de
points

└ Couleur du dinosaure
(dino_main.h)


```
/// Produit un dessin contenant une liste de segments, tous reliés.  
/// Les sommets doivent être dessiné avec XDino_TEXID_WHITE.  
///  
/// @param out Destination dans laquelle sont ajoutés les sommets.  
/// @param points Liste de points par lesquels la polyligne passe.  
/// @param width Epaisseur du trait, en pixels.  
/// @param color Couleur du trait.  
void GenVertices_Polyline(std::vector<jv::gpu::Vertex>& out, std::vector<Vec2> const& points,  
    float width, jv::util::Color color = Color_WHITE);
```

↓

pVBuf = CreateVertexBuffer();
Draw(pVBuf, nullptr);
DestroyVertexBuffer();

Garder 2 secondes de lasso derrière soi

Le lasso est défini par une suite de points pris chaque frame

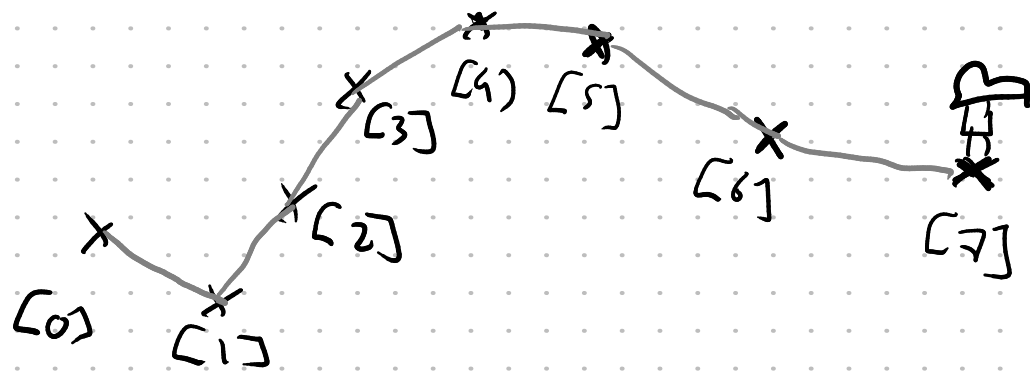
Le jeu tourne à 60 frames par seconde

60 frames $\approx$ 1 seconde

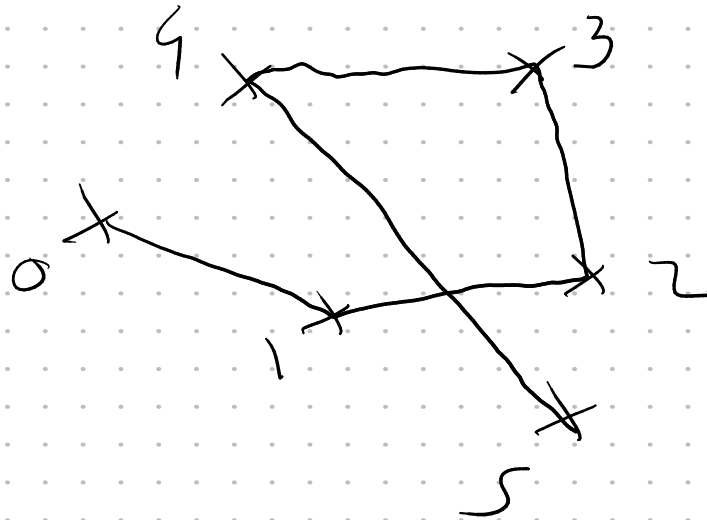
1 point / frame

2 secondes max

$\Rightarrow$ 120 points max dans le lasso



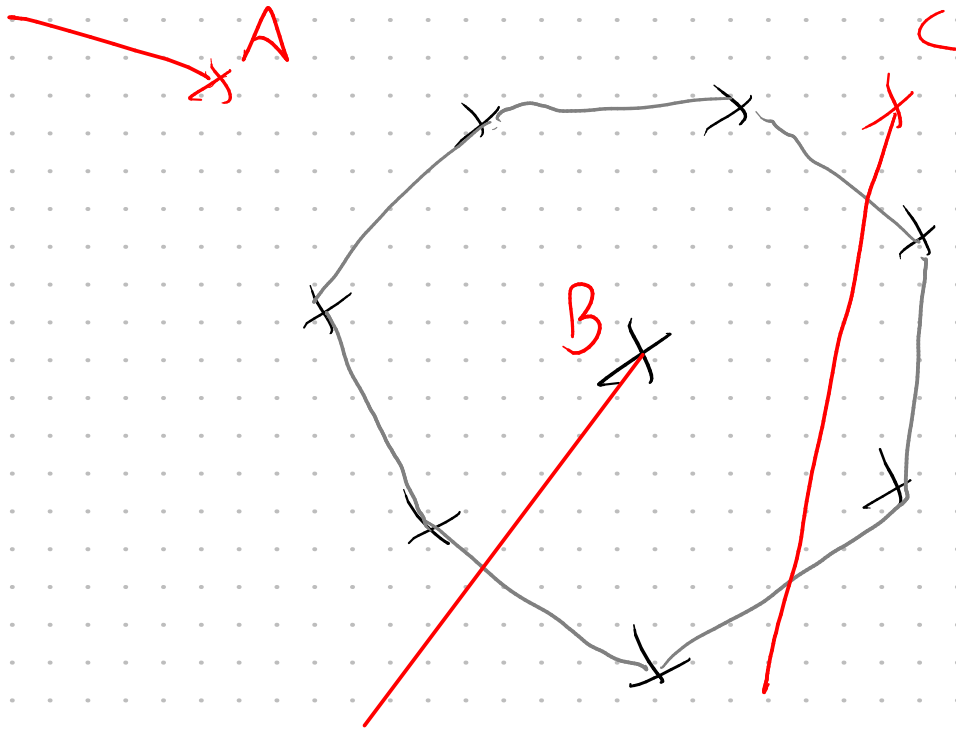
Lasso = suite de points
 = suite de segments
 (01, 12, 23, etc.)



Une boucle =
 Une suite de segments
 dont le premier et le
 dernier s'intersectent

→ dino-geometry.h

```
/// Vérifie si les deux segments [AB] et [CD] ont une intersection.  
bool IntersectSegment(Vec2 a, Vec2 b, Vec2 c, Vec2 d);
```



P appartient à la boucle?

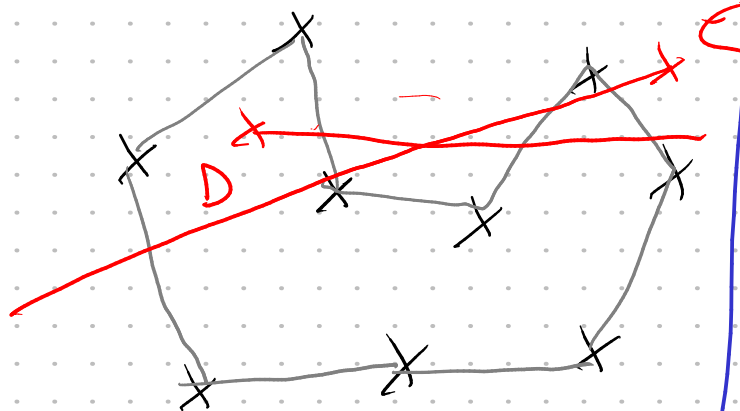
$A : 0 \rightarrow$ Extérieur

$B : 1$ intersection avec la
 $\rightarrow$ Intérieur contour

$C : 2 \rightarrow$ Extérieur

$D : 3 \rightarrow$ Intérieur

$E : 4 \rightarrow$ Extérieur



Check the intersection entre
 un segment $[P, E[0,1]]$

et les segments de la boucle

Pair $\rightarrow$ Extérieur

Impair $\rightarrow$ Intérieur